

9.1 节练习

练习 9.1: 对于下面的程序任务, `vector`、`deque` 和 `list` 哪种容器最为适合? 解释你的选择的理由。如果没有哪一种容器优于其他容器, 也请解释理由。

- (a) 读取固定数量的单词, 将它们按字典序插入到容器中。我们将在下一章中看到, 关联容器更适合这个问题。
- (b) 读取未知数量的单词, 总是将新单词插入到末尾。删除操作在头部进行。
- (c) 从一个文件读取未知数量的整数。将这些数排序, 然后将它们打印到标准输出。



9.2 容器库概览

容器类型上的操作形成了一种层次:

- 某些操作是所有容器类型都提供的 (参见表 9.2, 第 295 页)。
- 另外一些操作仅针对顺序容器 (参见表 9.3, 第 299 页)、关联容器 (参见表 11.7, 第 388 页) 或无序容器 (参见表 11.8, 第 395 页)。
- 还有一些操作只适用于一小部分容器。

329

在本节中, 我们将介绍对所有容器都适用的操作。本章剩余部分将聚焦于仅适用于顺序容器的操作。关联容器特有的操作将在第 11 章介绍。

一般来说, 每个容器都定义在一个头文件中, 文件名与类型名相同。即, `deque` 定义在头文件 `deque` 中, `list` 定义在头文件 `list` 中, 以此类推。容器均定义为模板类 (参见 3.3 节, 第 86 页)。例如对 `vector`, 我们必须提供额外信息来生成特定的容器类型。对大多数, 但不是所有容器, 我们还需要额外提供元素类型信息:

```
list<Sales_data>      // 保存 Sales_data 对象的 list
deque<double>         // 保存 double 的 deque
```

对容器可以保存的元素类型的限制

顺序容器几乎可以保存任意类型的元素。特别是, 我们可以定义一个容器, 其元素的类型是另一个容器。这种容器的定义与任何其他容器类型完全一样: 在尖括号中指定元素类型 (此种情况下, 是另一种容器类型):

```
vector<vector<string>> lines; // vector 的 vector
```

C++
11

此处 `lines` 是一个 `vector`, 其元素类型是 `string` 的 `vector`。



较旧的编译器可能需要在两个尖括号之间键入空格, 例如, `vector<vector<string> >`。

虽然我们可以在容器中保存几乎任何类型, 但某些容器操作对元素类型有其自己的特殊要求。我们可以为不支持特定操作需求的类型定义容器, 但这种情况下就只能使用那些没有特殊要求的容器操作了。

例如, 顺序容器构造函数的一个版本接受容器大小参数 (参见 3.3.1 节, 第 88 页), 它使用了元素类型的默认构造函数。但某些类没有默认构造函数。我们可以定义一个保存这种类型对象的容器, 但我们在构造这种容器时不能只传递给它一个元素数目参数:

```
// 假定 noDefault 是一个没有默认构造函数的类型
vector<noDefault> v1(10, init);           // 正确: 提供了元素初始化器
vector<noDefault> v2(10);                 // 错误: 必须提供一个元素初始化器
```

当后面介绍容器操作时，我们还会注意到每个容器操作对元素类型的其他限制。

表 9.2: 容器操作	
类型别名	
iterator	此容器类型的迭代器类型
const_iterator	可以读取元素，但不能修改元素的迭代器类型
size_type	无符号整数类型，足够保存此种容器类型最大可能容器的大小
difference_type	带符号整数类型，足够保存两个迭代器之间的距离
value_type	元素类型
reference	元素的左值类型；与 value_type&含义相同
const_reference	元素的 const 左值类型（即，const value_type&）
构造函数	
C c;	默认构造函数，构造空容器（array，参见第 301 页）
C c1(c2);	构造 c2 的拷贝 c1
C c(b, e);	构造 c，将迭代器 b 和 e 指定的范围内的元素拷贝到 c（array 不支持）
C c{a, b, c...};	列表初始化 c
赋值与 swap	
c1=c2	将 c1 中的元素替换为 c2 中元素
c1={a, b, c...}	将 c1 中的元素替换为列表中元素（不适用于 array）
a.swap(b)	交换 a 和 b 的元素
swap(a, b)	与 a.swap(b) 等价
大小	
c.size()	c 中元素的数目（不支持 forward_list）
c.max_size()	c 可保存的最大元素数目
c.empty()	若 c 中存储了元素，返回 false，否则返回 true
添加/删除元素（不适用于 array）	
注：在不同容器中，这些操作的接口都不同	
c.insert(args)	将 args 中的元素拷贝进 c
c.emplace(inits)	使用 inits 构造 c 中的一个元素
c.erase(args)	删除 args 指定的元素
c.clear()	删除 c 中的所有元素，返回 void
关系运算符	
==, !=	所有容器都支持相等（不等）运算符
<, <=, >, >=	关系运算符（无序关联容器不支持）
获取迭代器	
c.begin(),c.end()	返回指向 c 的首元素和尾元素之后位置的迭代器
c.cbegin(),c.cend()	返回 const_iterator

续表

反向容器的额外成员（不支持 `forward_list`）

<code>reverse_iterator</code>	按逆序寻址元素的迭代器
<code>const_reverse_iterator</code>	不能修改元素的逆序迭代器
<code>c.rbegin(), c.rend()</code>	返回指向 <code>c</code> 的尾元素和首元素之前位置的迭代器
<code>c.crbegin(), c.crend()</code>	返回 <code>const_reverse_iterator</code>

9.2 节练习

练习 9.2: 定义一个 `list` 对象，其元素类型是 `int` 的 `deque`。

331

9.2.1 迭代器



与容器一样，迭代器有着公共的接口：如果一个迭代器提供某个操作，那么所有提供相同操作的迭代器对这个操作的实现方式都是相同的。例如，标准容器类型上的所有迭代器都允许我们访问容器中的元素，而所有迭代器都是通过解引用运算符来实现这个操作的。类似的，标准库容器的所有迭代器都定义了递增运算符，从当前元素移动到下一个元素。

表 3.6（第 96 页）列出了容器迭代器支持的所有操作，其中有一个例外不符合公共接口特点——`forward_list` 迭代器不支持递减运算符（--）。表 3.7（第 99 页）列出了迭代器支持的算术运算，这些运算只能应用于 `string`、`vector`、`deque` 和 `array` 的迭代器。我们不能将它们用于其他任何容器类型的迭代器。

迭代器范围



迭代器范围的概念是标准库的基础。

一个迭代器范围（`iterator range`）由一对迭代器表示，两个迭代器分别指向同一个容器中的元素或者是尾元素之后的位置（`one past the last element`）。这两个迭代器通常被称为 `begin` 和 `end`，或者是 `first` 和 `last`（可能有些误导），它们标记了容器中元素的一个范围。

虽然第二个迭代器常常被称为 `last`，但这种叫法有些误导，因为第二个迭代器从来都不会指向范围中的最后一个元素，而是指向尾元素之后的位置。迭代器范围中的元素包含 `first` 所表示的元素以及从 `first` 开始直至 `last`（但不包含 `last`）之间的所有元素。

这种元素范围被称为左闭合区间（`left-inclusive interval`），其标准数学描述为

`[begin, end)`

表示范围自 `begin` 开始，于 `end` 之前结束。迭代器 `begin` 和 `end` 必须指向相同的容器。`end` 可以与 `begin` 指向相同的位置，但不能指向 `begin` 之前的位置。

对构成范围的迭代器的要求

如果满足如下条件，两个迭代器 `begin` 和 `end` 构成一个迭代器范：

- 它们指向同一个容器中的元素，或者是容器最后一个元素之后的位置，且
- 我们可以通过反复递增 `begin` 来到达 `end`。换句话说，`end` 不在 `begin` 之前。



编译器不会强制这些要求。确保程序符合这些约定是程序员的责任。

使用左闭合范围蕴含的编程假定

标准库使用左闭合范围是因为这种范围有三种方便的性质。假定 `begin` 和 `end` 构成一个合法的迭代器范围，则 ◀ 332

- 如果 `begin` 与 `end` 相等，则范围为空
- 如果 `begin` 与 `end` 不等，则范围至少包含一个元素，且 `begin` 指向该范围中的第一个元素
- 我们可以对 `begin` 递增若干次，使得 `begin==end`

这些性质意味着我们可以像下面的代码一样用一个循环来处理一个元素范围，而这是安全的：

```
while (begin != end) {
    *begin = val; // 正确：范围非空，因此 begin 指向一个元素
    ++begin;      // 移动迭代器，获取下一个元素
}
```

给定构成一个合法范围的迭代器 `begin` 和 `end`，若 `begin==end`，则范围为空。在此情况下，我们应该退出循环。如果范围不为空，`begin` 指向此非空范围的一个元素。因此，在 `while` 循环体中，可以安全地解引用 `begin`，因为 `begin` 必然指向一个元素。最后，由于每次循环对 `begin` 递增一次，我们确定循环最终会结束。

9.2.1 节练习

练习 9.3：构成迭代器范围的迭代器有何限制？

练习 9.4：编写函数，接受一对指向 `vector<int>` 的迭代器和一个 `int` 值。在两个迭代器指定的范围中查找给定的值，返回一个布尔值来指出是否找到。

练习 9.5：重写上一题的函数，返回一个迭代器指向找到的元素。注意，程序必须处理未找到给定值的情况。

练习 9.6：下面程序有何错误？你应该如何修改它？

```
list<int> lst1;
list<int>::iterator iter1 = lst1.begin(),
                    iter2 = lst1.end();
while (iter1 < iter2) /* ... */
```

9.2.2 容器类型成员

每个容器都定义了多个类型，如表 9.2 所示（第 295 页）。我们已经使用过其中三种：`size_type`（参见 3.2.2 节，第 79 页）、`iterator` 和 `const_iterator`（参见 3.4.1 节，第 97 页）。

除了已经使用过的迭代器类型，大多数容器还提供反向迭代器。简单地说，反向迭代器就是一种反向遍历容器的迭代器，与正向迭代器相比，各种操作的含义也都发生了颠倒。例如，对一个反向迭代器执行 `++` 操作，会得到上一个元素。我们将在 10.4.3 节（第 363 页）◀ 333

介绍更多关于反向迭代器的内容。

剩下的就是类型别名了，通过类型别名，我们可以在不了解容器中元素类型的情况下使用它。如果需要元素类型，可以使用容器的 `value_type`。如果需要元素类型的一个引用，可以使用 `reference` 或 `const_reference`。这些元素相关的类型别名在泛型编程中非常有用，我们将在 16 章中介绍相关内容。

为了使用这些类型，我们必须显式使用其类名：

```
// iter 是通过 list<string>定义的一个迭代器类型
list<string>::iterator iter;
// count 是通过 vector<int>定义的一个 difference_type 类型
vector<int>::difference_type count;
```

这些声明语句使用了作用域运算符（参见 1.2 节，第 7 页）来说明我们希望使用 `list<string>` 类的 `iterator` 成员及 `vector<int>` 类定义的 `difference_type`。

9.2.2 节练习

练习 9.7：为了索引 `int` 的 `vector` 中的元素，应该使用什么类型？

练习 9.8：为了读取 `string` 的 `list` 中的元素，应该使用什么类型？如果写入 `list`，又该使用什么类型？



9.2.3 begin 和 end 成员

`begin` 和 `end` 操作（参见 3.4.1 节，第 95 页）生成指向容器中第一个元素和尾元素之后位置的迭代器。这两个迭代器最常见的用途是形成一个包含容器中所有元素的迭代器范围。

如表 9.2（第 295 页）所示，`begin` 和 `end` 有多个版本：带 `r` 的版本返回反向迭代器（我们将在 10.4.3 节（第 363 页）中介绍相关内容）；以 `c` 开头的版本则返回 `const` 迭代器：

```
list<string> a = {"Milton", "Shakespeare", "Austen"};
auto it1 = a.begin(); // list<string>::iterator
auto it2 = a.rbegin(); // list<string>::reverse_iterator
auto it3 = a.cbegin(); // list<string>::const_iterator
auto it4 = a.crbegin(); // list<string>::const_reverse_iterator
```

不以 `c` 开头的函数都是被重载过的。也就是说，实际上有两个名为 `begin` 的成员。一个是 `const` 成员（参见 7.1.2 节，第 231 页），返回容器的 `const_iterator` 类型。另一个是非常量成员，返回容器的 `iterator` 类型。`rbegin`、`end` 和 `rend` 的情况类似。当我们对一个非常量对象调用这些成员时，得到的是返回 `iterator` 的版本。只有在对一个 `const` 对象调用这些函数时，才会得到一个 `const` 版本。与 `const` 指针和引用类似，可以将一个普通的 `iterator` 转换为对应的 `const_iterator`，但反之不行。

334

以 `c` 开头的版本是 C++ 新标准引入的，用以支持 `auto`（参见 2.5.2 节，第 61 页）与 `begin` 和 `end` 函数结合使用。过去，没有其他选择，只能显式声明希望使用哪种类型的迭代器：

```
// 显式指定类型
list<string>::iterator it5 = a.begin();
```

C++
11

```
list<string>::const_iterator it6 = a.begin();
// 是 iterator 还是 const_iterator 依赖于 a 的类型
auto it7 = a.begin(); // 仅当 a 是 const 时, it7 是 const_iterator
auto it8 = a.cbegin(); // it8 是 const_iterator
```

当 auto 与 begin 或 end 结合使用时, 获得的迭代器类型依赖于容器类型, 与我们想要如何使用迭代器毫不相干。但以 c 开头的版本还是可以获得 const_iterator 的, 而不管容器的类型是什么。

 **Best Practices**

当不需要写访问时, 应使用 cbegin 和 cend。

9.2.3 节练习

练习 9.9: begin 和 cbegin 两个函数有什么不同?

练习 9.10: 下面 4 个对象分别是什么类型?

```
vector<int> v1;
const vector<int> v2;
auto it1 = v1.begin(), it2 = v2.begin();
auto it3 = v1.cbegin(), it4 = v2.cbegin();
```

9.2.4 容器定义和初始化



每个容器类型都定义了一个默认构造函数 (参见 7.1.4 节, 第 236 页)。除 array 之外, 其他容器的默认构造函数都会创建一个指定类型的空容器, 且都可以接受指定容器大小和元素初始值的参数。

表 9.3: 容器定义和初始化	
C c;	默认构造函数。如果 C 是一个 array, 则 c 中元素按默认方式初始化; 否则 c 为空
C c1(c2)	c1 初始化为 c2 的拷贝。c1 和 c2 必须是相同类型 (即, 它们必须是相同的容器类型, 且保存的是相同的元素类型; 对于 array 类型, 两者还必须具有相同大小)
C c1=c2	
C c{a,b,c,...}	c 初始化为初始化列表中元素的拷贝。列表中元素的类型必须与 c 的元素类型相容。对于 array 类型, 列表中元素数目必须等于或小于 array 的大小, 任何遗漏的元素都进行值初始化 (参见 3.3.1 节, 第 88 页)
C c={a,b,c,...}	
C c(b,e)	c 初始化为迭代器 b 和 e 指定范围中的元素的拷贝。范围中元素的类型必须与 c 的元素类型相容 (array 不适用)
只有顺序容器 (不包括 array) 的构造函数才能接受大小参数	
C seq(n)	seq 包含 n 个元素, 这些元素进行了值初始化; 此构造函数是 explicit 的 (参见 7.5.4 节, 第 265 页)。(string 不适用)
C seq(n,t)	seq 包含 n 个初始化为值 t 的元素

将一个容器初始化为另一个容器的拷贝

将一个新容器创建为另一个容器的拷贝的方法有两种: 可以直接拷贝整个容器, 或者

(array 除外) 拷贝由一个迭代器对指定的元素范围。

为了创建一个容器为另一个容器的拷贝, 两个容器的类型及其元素类型必须匹配。不过, 当传递迭代器参数来拷贝一个范围时, 就不要求容器类型是相同的了。而且, 新容器和原容器中的元素类型也可以不同, 只要能将要拷贝的元素转换 (参见 4.11 节, 第 141 页) 为要初始化的容器的元素类型即可。

335

```
// 每个容器有三个元素, 用给定的初始化器进行初始化
list<string> authors = {"Milton", "Shakespeare", "Austen"};
vector<const char*> articles = {"a", "an", "the"};

list<string> list2(authors);      // 正确: 类型匹配
deque<string> authList(authors); // 错误: 容器类型不匹配
vector<string> words(articles);  // 错误: 容器类型必须匹配
// 正确: 可以将 const char* 元素转换为 string
forward_list<string> words(articles.begin(), articles.end());
```



将一个容器初始化为另一个容器的拷贝时, 两个容器的容器类型和元素类型都必须相同。

接受两个迭代器参数的构造函数用这两个迭代器表示我们想要拷贝的一个元素范围。与以往一样, 两个迭代器分别标记想要拷贝的第一个元素和尾元素之后的位置。新容器的大小与范围中元素的数目相同。新容器中的每个元素都用范围中对应元素的值进行初始化。

由于两个迭代器表示一个范围, 因此可以使用这种构造函数来拷贝一个容器中的子序列。例如, 假定迭代器 `it` 表示 `authors` 中的一个元素, 我们可以编写如下代码

```
// 拷贝元素, 直到 (但不包括) it 指向的元素
deque<string> authList(authors.begin(), it);
```

336

列表初始化

C++
11

在新标准中, 我们可以对一个容器进行列表初始化 (参见 3.3.1 节, 第 88 页)

```
// 每个容器有三个元素, 用给定的初始化器进行初始化
list<string> authors = {"Milton", "Shakespeare", "Austen"};
vector<const char*> articles = {"a", "an", "the"};
```

当这样做时, 我们就显式地指定了容器中每个元素的值。对于除 `array` 之外的容器类型, 初始化列表还隐含地指定了容器的大小: 容器将包含与初始值一样多的元素。

与顺序容器大小相关的构造函数

除了与关联容器相同的构造函数外, 顺序容器 (array 除外) 还提供另一个构造函数, 它接受一个容器大小和一个 (可选的) 元素初始值。如果我们不提供元素初始值, 则标准库会创建一个值初始化器 (参见 3.3.1 节, 第 88 页):

```
vector<int> ivec(10, -1);      // 10 个 int 元素, 每个都初始化为 -1
list<string> svec(10, "hi!");  // 10 个 strings; 每个都初始化为 "hi!"
forward_list<int> ivec(10);    // 10 个元素, 每个都初始化为 0
deque<string> svec(10);        // 10 个元素, 每个都是空 string
```


如果元素类型是内置类型或者是具有默认构造函数（参见 9.2 节，第 294 页）的类类型，可以只为构造函数提供一个容器大小参数。如果元素类型没有默认构造函数，除了大小参数外，还必须指定一个显式的元素初始值。



只有顺序容器的构造函数才接受大小参数，关联容器并不支持。

标准库 array 具有固定大小

与内置数组一样，标准库 array 的大小也是类型的一部分。当定义一个 array 时，除了指定元素类型，还要指定容器大小：

```
array<int, 42>           // 类型为：保存 42 个 int 的数组
array<string, 10>        // 类型为：保存 10 个 string 的数组
```

为了使用 array 类型，我们必须同时指定元素类型和大小：

```
array<int, 10>::size_type i;    // 数组类型包括元素类型和大小
array<int>::size_type j;        // 错误：array<int>不是一个类型
```

由于大小是 array 类型的一部分，array 不支持普通的容器构造函数。这些构造函数都会确定容器的大小，要么隐式地，要么显式地。而允许用户向一个 array 构造函数传递大小参数，最好情况下也是多余的，而且容易出错。

array 大小固定的特性也影响了它所定义的构造函数的行为。与其他容器不同，一个默认构造的 array 是非空的：它包含了与其大小一样多的元素。这些元素都被默认初始化（参见 2.2.1 节，第 40 页），就像一个内置数组（参见 3.5.1 节，第 102 页）中的元素那样。如果我们对 array 进行列表初始化，初始值的数目必须等于或小于 array 的大小。如果初始值数目小于 array 的大小，则它们被用来初始化 array 中靠前的元素，所有剩余元素都会进行值初始化（参见 3.3.1 节，第 88 页）。在这两种情况下，如果元素类型是一个类类型，那么该类必须有一个默认构造函数，以使值初始化能够进行：

◀ 337

```
array<int, 10> ia1;           // 10 个默认初始化的 int
array<int, 10> ia2 = {0,1,2,3,4,5,6,7,8,9}; // 列表初始化
array<int, 10> ia3 = {42};    // ia3[0] 为 42，剩余元素为 0
```

值得注意的是，虽然我们不能对内置数组类型进行拷贝或对象赋值操作（参见 3.5.1 节，第 102 页），但 array 并无此限制：

```
int digs[10] = {0,1,2,3,4,5,6,7,8,9};
int cpy[10] = digs;           // 错误：内置数组不支持拷贝或赋值
array<int, 10> digits = {0,1,2,3,4,5,6,7,8,9};
array<int, 10> copy = digits; // 正确：只要数组类型匹配即合法
```

与其他容器一样，array 也要求初始值的类型必须与要创建的容器类型相同。此外，array 还要求元素类型和大小也都一样，因为大小是 array 类型的一部分。

9.2.4 节练习

练习 9.11: 对 6 种创建和初始化 vector 对象的方法，每一种都给出一个实例。解释每个 vector 包含什么值。

练习 9.12: 对于接受一个容器创建其拷贝的构造函数，和接受两个迭代器创建拷贝的构造函数，解释它们的不同。

练习 9.13: 如何从一个 `list<int>` 初始化一个 `vector<double>`? 从一个 `vector<int>` 又如何创建? 编写代码验证你的答案。

9.2.5 赋值和 swap

表 9.4 中列出的与赋值相关的运算符可用于所有容器。赋值运算符将其左边容器中的全部元素替换为右边容器中元素的拷贝：

```
c1 = c2;           // 将 c1 的内容替换为 c2 中元素的拷贝
c1 = {a,b,c};      // 赋值后, c1 大小为 3
```

第一个赋值运算后, 左边容器将与右边容器相等。如果两个容器原来大小不同, 赋值运算后两者的大小都与右边容器的原大小相同。第二个赋值运算后, `c1` 的 `size` 变为 3, 即花括号列表中值的数目。

338

与内置数组不同, 标准库 `array` 类型允许赋值。赋值号左右两边的运算对象必须具有相同的类型:

```
array<int, 10> a1 = {0,1,2,3,4,5,6,7,8,9};
array<int, 10> a2 = {0}; // 所有元素值均为 0
a1 = a2; // 替换 a1 中的元素
a2 = {0}; // 错误: 不能将一个花括号列表赋予数组
```

由于右边运算对象的大小可能与左边运算对象的大小不同, 因此 `array` 类型不支持 `assign`, 也不允许用花括号包围的值列表进行赋值。

表 9.4: 容器赋值运算	
<code>c1=c2</code>	将 <code>c1</code> 中的元素替换为 <code>c2</code> 中元素的拷贝。 <code>c1</code> 和 <code>c2</code> 必须具有相同的类型
<code>c={a,b,c...}</code>	将 <code>c1</code> 中元素替换为初始化列表中元素的拷贝 (<code>array</code> 不适用)
<code>swap(c1,c2)</code> <code>c1.swap(c2)</code>	交换 <code>c1</code> 和 <code>c2</code> 中的元素。 <code>c1</code> 和 <code>c2</code> 必须具有相同的类型。 <code>swap</code> 通常比从 <code>c2</code> 向 <code>c1</code> 拷贝元素快得多
<code>assign</code> 操作不适用于关联容器和 <code>array</code>	
<code>seq.assign(b,e)</code>	将 <code>seq</code> 中的元素替换为迭代器 <code>b</code> 和 <code>e</code> 所表示的范围中的元素。迭代器 <code>b</code> 和 <code>e</code> 不能指向 <code>seq</code> 中的元素
<code>seq.assign(il)</code>	将 <code>seq</code> 中的元素替换为初始化列表 <code>il</code> 中的元素
<code>seq.assign(n,t)</code>	将 <code>seq</code> 中的元素替换为 <code>n</code> 个值为 <code>t</code> 的元素
 WARNING	赋值相关运算会导致指向左边容器内部的迭代器、引用和指针失效。而 <code>swap</code> 操作将容器内容交换不会导致指向容器的迭代器、引用和指针失效 (容器类型为 <code>array</code> 和 <code>string</code> 的情况除外)。

使用 assign (仅顺序容器)

赋值运算符要求左边和右边的运算对象具有相同的类型。它将右边运算对象中所有元素拷贝到左边运算对象中。顺序容器 (`array` 除外) 还定义了一个名为 `assign` 的成员, 允许我们从一个不同但相容的类型赋值, 或者从容器的一个子序列赋值。`assign` 操作作用参数所指定的元素 (的拷贝) 替换左边容器中的所有元素。例如, 我们可以用 `assign` 实现将一个 `vector` 中的一段 `char *` 值赋予一个 `list` 中的 `string`:

```
list<string> names;
vector<const char*> oldstyle;
names = oldstyle; // 错误: 容器类型不匹配
// 正确: 可以将 const char* 转换为 string
names.assign(oldstyle.cbegin(), oldstyle.cend());
```

这段代码中对 `assign` 的调用将 `names` 中的元素替换为迭代器指定的范围中的元素的拷贝。339
`assign` 的参数决定了容器中将有有多少个元素以及它们的值都是什么。



WARNING 由于其旧元素被替换, 因此传递给 `assign` 的迭代器不能指向调用 `assign` 的容器。

`assign` 的第二个版本接受一个整型值和一个元素值。它用指定数目且具有相同给定值的元素替换容器中原有的元素:

```
// 等价于 slist1.clear();
// 后跟 slist1.insert(slist1.begin(), 10, "Hiya!");
list<string> slist1(1); // 1 个元素, 为空 string
slist1.assign(10, "Hiya!"); // 10 个元素, 每个都是 "Hiya!"
```

使用 swap

`swap` 操作交换两个相同类型容器的内容。调用 `swap` 之后, 两个容器中的元素将会交换:

```
vector<string> svec1(10); // 10 个元素的 vector
vector<string> svec2(24); // 24 个元素的 vector
swap(svec1, svec2);
```

调用 `swap` 后, `svec1` 将包含 24 个 `string` 元素, `svec2` 将包含 10 个 `string`。除 `array` 外, 交换两个容器内容的操作保证会很快——元素本身并未交换, `swap` 只是交换了两个容器的内部数据结构。



除 `array` 外, `swap` 不对任何元素进行拷贝、删除或插入操作, 因此可以保证在常数时间内完成。

元素不会被移动的事实意味着, 除 `string` 外, 指向容器的迭代器、引用和指针在 `swap` 操作之后都不会失效。它们仍指向 `swap` 操作之前所指向的那些元素。但是, 在 `swap` 之后, 这些元素已经属于不同的容器了。例如, 假定 `iter` 在 `swap` 之前指向 `svec1[3]` 的 `string`, 那么在 `swap` 之后它指向 `svec2[3]` 的元素。与其他容器不同, 对一个 `string` 调用 `swap` 会导致迭代器、引用和指针失效。

与其他容器不同, `swap` 两个 `array` 会真正交换它们的元素。因此, 交换两个 `array` 所需的时间与 `array` 中元素的数目成正比。

因此, 对于 `array`, 在 `swap` 操作之后, 指针、引用和迭代器所绑定的元素保持不变, 但元素值已经与另一个 `array` 中对应元素的值进行了交换。

在新标准库中, 容器既提供成员函数版本的 `swap`, 也提供非成员版本的 `swap`。而早期标准库版本只提供成员函数版本的 `swap`。非成员版本的 `swap` 在泛型编程中是非常重要的。统一使用非成员版本的 `swap` 是一个好习惯。

340

9.2.5 节练习

练习 9.14: 编写程序, 将一个 `list` 中的 `char *` 指针 (指向 C 风格字符串) 元素赋值给一个 `vector` 中的 `string`。



9.2.6 容器大小操作

除了一个例外, 每个容器类型都有三个与大小相关的操作。成员函数 `size` (参见 3.2.2 节, 第 78 页) 返回容器中元素的数目; `empty` 当 `size` 为 0 时返回布尔值 `true`, 否则返回 `false`; `max_size` 返回一个大于或等于该类型容器所能容纳的最大元素数的值。`forward_list` 支持 `max_size` 和 `empty`, 但不支持 `size`, 原因我们将在下一节解释。

9.2.7 关系运算符

每个容器类型都支持相等运算符 (`==` 和 `!=`); 除了无序关联容器外的所有容器都支持关系运算符 (`>`、`>=`、`<`、`<=`)。关系运算符左右两边的运算对象必须是相同类型的容器, 且必须保存相同类型的元素。即, 我们只能将一个 `vector<int>` 与另一个 `vector<int>` 进行比较, 而不能将一个 `vector<int>` 与一个 `list<int>` 或一个 `vector<double>` 进行比较。

比较两个容器实际上是进行元素的逐对比较。这些运算符的工作方式与 `string` 的关系运算 (参见 3.2.2 节, 第 79 页) 类似:

- 如果两个容器具有相同大小且所有元素都两两对应相等, 则这两个容器相等; 否则两个容器不等。
- 如果两个容器大小不同, 但较小容器中每个元素都等于较大容器中的对应元素, 则较小容器小于较大容器。
- 如果两个容器都不是另一个容器的前缀子序列, 则它们的比较结果取决于第一个不相等的元素的比较结果。

下面的例子展示了这些关系运算符是如何工作的:

```
vector<int> v1 = { 1, 3, 5, 7, 9, 12 };
vector<int> v2 = { 1, 3, 9 };
vector<int> v3 = { 1, 3, 5, 7 };
vector<int> v4 = { 1, 3, 5, 7, 9, 12 };
v1 < v2 // true; v1 和 v2 在元素[2]处不同: v1[2]小于等于 v2[2]
v1 < v3 // false; 所有元素都相等, 但 v3 中元素数目更少
v1 == v4 // true; 每个元素都相等, 且 v1 和 v4 大小相同
v1 == v2 // false; v2 元素数目比 v1 少
```

341

容器的关系运算符使用元素的关系运算符完成比较



只有当其元素类型也定义了相应的比较运算符时, 我们才可以使用关系运算符来比较两个容器。

容器的相等运算符实际上是使用元素的 `==` 运算符实现比较的, 而其他关系运算符是使用元素的 `<` 运算符。如果元素类型不支持所需运算符, 那么保存这种元素的容器就不能使用相应的关系运算。例如, 我们在第 7 章中定义的 `Sales_data` 类型并未定义 `==` 和 `<` 运算。因此, 就不能比较两个保存 `Sales_data` 元素的容器:

```
vector<Sales_data> storeA, storeB;
if (storeA < storeB) // 错误: Sales_data 没有<运算符
```

9.2.7 节练习

练习 9.15: 编写程序, 判定两个 `vector<int>` 是否相等。

练习 9.16: 重写上一题的程序, 比较一个 `list<int>` 中的元素和一个 `vector<int>` 中的元素。

练习 9.17: 假定 `c1` 和 `c2` 是两个容器, 下面的比较操作有何限制 (如果有的话)?

```
if (c1 < c2)
```

9.3 顺序容器操作

顺序容器和关联容器的不同之处在于两者组织元素的方式。这些不同之处直接关系到元素如何存储、访问、添加以及删除。上一节介绍了所有容器都支持的操作 (罗列于表 9.2 (第 295 页))。本章剩余部分将介绍顺序容器所特有的操作。

9.3.1 向顺序容器添加元素



除 `array` 外, 所有标准库容器都提供灵活的内存管理。在运行时可以动态添加或删除元素来改变容器大小。表 9.5 列出了向顺序容器 (非 `array`) 添加元素的操作。

表 9.5: 向顺序容器添加元素的操作

这些操作会改变容器的大小; `array` 不支持这些操作。

`forward_list` 有自己专有版本的 `insert` 和 `emplace`; 参见 9.3.4 节 (第 312 页)。

`forward_list` 不支持 `push_back` 和 `emplace_back`。

`vector` 和 `string` 不支持 `push_front` 和 `emplace_front`。

`c.push_back(t)` 在 `c` 的尾部创建一个值为 `t` 或由 `args` 创建的元素。返回 `void`
`c.emplace_back(args)`

`c.push_front(t)` 在 `c` 的头部创建一个值为 `t` 或由 `args` 创建的元素。返回 `void`
`c.emplace_front(args)`

`c.insert(p, t)` 在迭代器 `p` 指向的元素之前创建一个值为 `t` 或由 `args` 创建的元素。返回指向新添加的元素的迭代器
`c.emplace(p, args)`

`c.insert(p, n, t)` 在迭代器 `p` 指向的元素之前插入 `n` 个值为 `t` 的元素。返回指向新添加的第一个元素的迭代器; 若 `n` 为 0, 则返回 `p`

`c.insert(p, b, e)` 将迭代器 `b` 和 `e` 指定的范围内的元素插入到迭代器 `p` 指向的元素之前。`b` 和 `e` 不能指向 `c` 中的元素。返回指向新添加的第一个元素的迭代器; 若范围为空, 则返回 `p`

`c.insert(p, il)` `il` 是一个花括号包围的元素值列表。将这些给定值插入到迭代器 `p` 指向的元素之前。返回指向新添加的第一个元素的迭代器; 若列表为空, 则返回 `p`



向一个 `vector`、`string` 或 `deque` 插入元素会使所有指向容器的迭代器、引用和指针失效。